

Docker na Prática — Do Zero ao Projeto em Produção

Um guia didático usando o projeto DevOps World como exemplo real

Gerado especialmente para seu estudo

27 de agosto de 2026

Sumário

1	Prefácio	3
2	O que é Docker — A Ideia Central	4
2.1	O problema que o Docker resolve	4
2.2	Container versus Máquina Virtual	4
2.3	Os três conceitos fundamentais	4
3	O Dockerfile — A Receita da Imagem	6
3.1	O que é e para que serve	6
3.2	O Dockerfile do Backend (Node.js)	6
3.2.1	O conceito de Multi-Stage Build	8
4	O Docker Compose — O Maestro da Orquestra	9
4.1	Por que o Docker Compose existe	9
4.2	A estrutura do arquivo docker-compose.yml	9
4.3	O docker-compose.yml do Projeto — Linha a Linha	9
4.3.1	Serviço 1 — MySQL	10
4.3.2	Serviço 2 — Backend (Node.js)	12
4.3.3	Serviço 3 — Frontend (NGINX)	14
4.3.4	Volumes e Redes	16
5	Redes no Docker — Como os Containers se Comunicam	18
5.1	O problema sem rede	18
5.2	A rede bridge	18
5.3	Visualizando a comunicação	18
6	Volumes — Como os Dados Sobrevivem	20
6.1	O problema da efemeridade	20
6.2	Dois tipos de volume no projeto	20
6.2.1	Volume Nomeado — para dados do banco	20
6.2.2	Bind Mount — para arquivos do projeto	20
7	Ordem de Inicialização e Healthchecks	21
7.1	O problema do “quem sobe primeiro”	21

7.2	A solução: healthcheck + condition	21
7.3	A cadeia de dependências do projeto	21
8	Comandos Essenciais do Docker Compose	23
8.1	Subindo e parando	23
8.2	Monitorando	23
8.3	Acessando containers	24
8.4	Inspecionando	24
8.5	Limpeza	24
9	Conectando ao Banco de Dados — Como Funciona no Projeto	26
9.1	A jornada de uma requisição	26
9.2	Como o backend se conecta ao MySQL	26
9.3	Como o banco é inicializado automaticamente	27
10	Variáveis de Ambiente — A Forma Segura de Configurar	28
10.1	Por que não colocar credenciais no código	28
10.2	A solução: variáveis de ambiente	28
10.3	O arquivo .env.example	28
11	O nginx.conf — Como o NGINX Integra Tudo	30
12	Fluxo Completo — Do docker compose up ao Site Funcionando	32
13	Resumo dos Conceitos	33
14	Referência Rápida de Comandos	34
15	Próximos Tópicos para Estudar	35

1 Prefácio

Este documento foi criado para te ensinar Docker a partir de um projeto **real e funcional** — o **DevOps World**, uma aplicação web completa que já está rodando em produção com três containers: banco de dados MySQL, API Node.js e servidor NGINX.

A ideia aqui não é te dar uma lista seca de comandos. A ideia é te contar **a história de como os containers funcionam**, como eles se enxergam, como eles conversam entre si, e como o Docker Compose organiza tudo isso como um maestro regendo uma orquestra.

Ao final deste documento você vai entender:

- O que é um container e por que ele existe
- O que é uma imagem Docker e como ela é construída
- Como o Dockerfile descreve a receita de uma imagem
- Como o Docker Compose orquestra múltiplos containers
- Como conectar um backend a um banco de dados dentro do Docker
- Os comandos essenciais para o dia a dia
- Como funciona a rede interna entre containers
- Como os volumes persistem seus dados

Vamos usar o código real do projeto como exemplo em cada passo.

2 O que é Docker — A Ideia Central

2.1 O problema que o Docker resolve

Imagine que você desenvolveu uma aplicação Node.js no seu computador. Funciona perfeitamente. Aí você manda para um colega testar e ele diz: *“Não roda aqui.”* Você investiga e descobre que ele tem uma versão diferente do Node.js, uma variável de ambiente diferente, ou uma biblioteca do sistema operacional faltando.

Esse problema tem um nome clássico no mundo do desenvolvimento: **“na minha máquina funciona”**.

O Docker resolve isso com um conceito simples: **empacotar a aplicação junto com tudo que ela precisa para rodar** — o código, o runtime, as bibliotecas, as configurações — em uma unidade chamada **container**.

Um container é como uma caixa lacrada. Dentro dela, o ambiente é sempre o mesmo, não importa se está rodando no seu notebook, no servidor de produção, ou na máquina de um colega.

2.2 Container versus Máquina Virtual

Uma dúvida comum é: *“Isso não é a mesma coisa que uma máquina virtual?”*

Não exatamente. A diferença está em como cada um isola o ambiente:

Uma **máquina virtual** simula um computador inteiro — ela tem seu próprio sistema operacional completo, seu próprio kernel, seus próprios drivers. É pesada: ocupa gigabytes de disco e leva minutos para iniciar.

Um **container** compartilha o kernel do sistema operacional host, mas isola o processo em um namespace separado. É leve: ocupa megabytes e inicia em segundos.

Para o seu dia a dia como desenvolvedor, pense assim:

Uma VM é um apartamento inteiro com sua própria estrutura elétrica e hidráulica. Um container é um quarto dentro de um apartamento — compartilha a estrutura do prédio, mas tem sua própria porta e privacidade.

2.3 Os três conceitos fundamentais

Antes de entrar no código, você precisa ter esses três conceitos na cabeça:

Imagem é o molde. É um arquivo somente leitura que contém o sistema operacional base, as dependências e o código da sua aplicação. Pense nela como uma foto tirada do ambiente em um momento específico.

Container é a imagem em execução. Você pode criar dezenas de containers a partir da mesma imagem, cada um rodando de forma independente.

Dockerfile é a receita para construir a imagem. É um arquivo de texto com instruções passo a passo:

“Comece com o Ubuntu, instale o Node.js versão 20, copie meu código, execute esse comando.”

3 O Dockerfile — A Receita da Imagem

3.1 O que é e para que serve

O Dockerfile é um arquivo de texto simples que fica na raiz do seu projeto. Cada linha é uma instrução que diz ao Docker como montar a imagem.

Quando você executa `docker build`, o Docker lê esse arquivo de cima para baixo, executa cada instrução, e ao final empacota tudo em uma imagem pronta para uso.

3.2 O Dockerfile do Backend (Node.js)

Este é o Dockerfile do backend do projeto DevOps World. Vamos ler linha por linha:

```
# -- Estágio 1: instalação das dependências -----  
# Usamos a instrução AS para nomear este estágio de "deps"  
# node:20-alpine = Node.js versão 20 sobre Alpine Linux  
# Alpine é uma distribuição Linux minúscula (~5MB), ideal para containers  
FROM node:20-alpine AS deps  
  
# Define o diretório de trabalho dentro do container  
# A partir daqui, todos os comandos serão executados dentro de /app  
WORKDIR /app  
  
# Copia apenas o package.json para o container  
# Por que só o package.json e não tudo?  
# Porque o Docker tem um sistema de cache inteligente:  
# se o package.json não mudou, ele reaproveita as dependências  
# instaladas na última vez, sem precisar rodar npm install de novo  
COPY package.json ./  
  
# Instala as dependências de produção (sem as de desenvolvimento)  
# --omit=dev significa: não instale nodemon, jest e outras ferramentas de dev  
RUN npm install --omit=dev  
  
# -- Estágio 2: imagem final de produção -----
```

```

# Começa do zero com a mesma imagem base limpa
# Isso é chamado de "multi-stage build":
# o estágio 1 foi só para instalar dependências
# o estágio 2 é a imagem final, sem ferramentas de build desnecessárias
FROM node:20-alpine AS runner

WORKDIR /app

# Cria um grupo e um usuário sem privilégios de root
# Por segurança: um container nunca deve rodar como root
# Se alguém invadir a aplicação, não terá poderes de administrador
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Copia apenas as dependências já instaladas do estágio anterior
# O --from=deps diz: busque isso do estágio chamado "deps"
COPY --from=deps /app/node_modules ./node_modules

# Agora copia o restante do código fonte
COPY . .

# Remove arquivos sensíveis que não devem estar na imagem final
RUN rm -f .env .env.example

# Muda o usuário ativo para o usuário sem privilégios criado acima
USER appuser

# Documenta que a aplicação usa a porta 3000
# EXPOSE não abre a porta sozinho - é mais uma documentação
# A porta real é configurada no Docker Compose
EXPOSE 3000

# Define uma variável de ambiente padrão
ENV NODE_ENV=production

```

```

# Configura o healthcheck: o Docker vai verificar periodicamente
# se o container está saudável executando este comando
# --interval=30s → verifica a cada 30 segundos
# --timeout=5s → se não responder em 5s, considera falha
# --start-period=15s → aguarda 15s antes da primeira verificação (tempo de boot)
# --retries=3 → 3 falhas consecutivas = container "unhealthy"
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
  CMD wget -q0- http://localhost:3000/health || exit 1

# O comando que inicia a aplicação quando o container sobe
CMD ["node", "server.js"]

```

3.2.1 O conceito de Multi-Stage Build

Repare que o Dockerfile tem dois blocos FROM. Isso é chamado de **multi-stage build** (build em múltiplos estágios) e é uma das técnicas mais importantes para criar imagens enxutas.

Por que fazer isso?

Para instalar dependências do Node.js, você precisa do **npm**. Mas para *rodar* a aplicação, você não precisa do **npm** — só precisa dos arquivos **node_modules** que ele gerou.

Com dois estágios: - Estágio 1: tem o **npm**, instala as dependências, faz o trabalho pesado - Estágio 2: copia só o resultado (**node_modules** + código), sem trazer o **npm** junto

O resultado é uma imagem final muito menor. Em projetos grandes, isso pode reduzir o tamanho da imagem de 800MB para menos de 150MB.

4 O Docker Compose — O Maestro da Orquestra

4.1 Por que o Docker Compose existe

Até aqui você aprendeu a criar um container de uma única aplicação. Mas aplicações reais raramente rodam sozinhas. O DevOps World, por exemplo, precisa de:

1. Um banco de dados MySQL
2. Uma API Node.js que fala com o banco
3. Um NGINX que serve o frontend e encaminha as chamadas de API

Se você fosse gerenciar esses três containers manualmente, precisaria: - Criar uma rede Docker para eles se comunicarem - Iniciar o MySQL primeiro e esperar ele estar pronto - Só então iniciar o Node.js - Só então iniciar o NGINX - Passar variáveis de ambiente para cada um - Criar volumes para persistir os dados do banco - Lembrar de todos os parâmetros sempre que quiser subir tudo

Isso seria uma tortura. O **Docker Compose** resolve exatamente isso: você descreve toda essa configuração em um único arquivo `docker-compose.yml` e sobe tudo com um comando.

4.2 A estrutura do arquivo docker-compose.yml

O arquivo usa o formato **YAML** — uma linguagem de configuração baseada em indentação (espaços, não tabs). A hierarquia é definida pelo recuo do texto.

A estrutura principal tem três seções:

```
services:    # Os containers que vão rodar
volumes:     # Os volumes para persistir dados
networks:    # As redes de comunicação entre containers
```

4.3 O docker-compose.yml do Projeto — Linha a Linha

Agora vamos ler o arquivo completo do projeto com explicação detalhada de cada bloco:

```
# As linhas que começam com # são comentários - o Docker ignora
# Não existe mais o campo "version:" no Compose v2.
# Antes era obrigatório (version: "3.9"), hoje é obsoleto e foi removido.

# =====
# SEÇÃO: services
# Aqui você declara cada container que fará parte da sua aplicação.
```

```
# Cada entrada dentro de "services:" é um serviço (= um container).
# =====
services:
```

4.3.1 Serviço 1 — MySQL

```
# O nome "mysql" é o identificador deste serviço.
# Outros containers vão usar EXATAMENTE este nome para se comunicar.
# Exemplo: o backend se conecta ao banco usando DB_HOST=mysql
mysql:

# "image:" diz ao Docker para usar uma imagem pronta do Docker Hub
# em vez de construir uma. mysql:8.4 significa:
# - Imagem oficial do MySQL
# - Versão 8.4 (versão específica, não "latest")
# IMPORTANTE: sempre use versões específicas em produção.
# Se você usar "latest", uma atualização automática pode quebrar tudo.
image: mysql:8.4

# Nome amigável do container quando você listar com "docker ps"
# Sem isso, o Docker gera um nome aleatório como "devops-site_mysql_1"
container_name: devops-site-mysql

# Variáveis de ambiente passadas para dentro do container.
# A imagem oficial do MySQL lê essas variáveis na primeira inicialização
# para configurar as credenciais e criar o banco automaticamente.
environment:
    MYSQL_ROOT_PASSWORD: root          # Senha do usuário root (admin total)
    MYSQL_DATABASE:      devops_site   # Nome do banco criado automaticamente
    MYSQL_USER:          devops        # Usuário de aplicação (não root)
    MYSQL_PASSWORD:      devops        # Senha deste usuário

# Mapeamento de portas: "porta_no_host:porta_no_container"
# Isso expõe a porta 3306 do container na porta 3306 da sua máquina.
```

```

# Permite que ferramentas externas (DBeaver, TablePlus) se conectem.
# Se você não quiser expor ao mundo externo, remova esta linha -
# os outros containers ainda se comunicam pela rede interna.
ports:
  - "3306:3306"

# Volumes: como persistir dados além do ciclo de vida do container.
# CONCEITO CRÍTICO: quando um container morre, todos os dados dentro
# dele somem. Para dados que precisam sobreviver (como um banco de dados),
# usamos volumes.
volumes:
  # Volume nomeado: "mysql_data" → "/var/lib/mysql" dentro do container
  # mysql_data é definido na seção "volumes:" lá embaixo no arquivo.
  # O Docker cria uma pasta gerenciada em /var/lib/docker/volumes/
  # e monta no caminho indicado dentro do container.
  # Os dados do banco ficam em /var/lib/mysql - com este volume,
  # eles sobrevivem mesmo que o container seja destruído.
  - mysql_data:/var/lib/mysql

  # Bind mount: mapeia um arquivo/pasta do HOST para dentro do container.
  # "./db/init.sql" (pasta do projeto) → caminho especial do MySQL
  # A imagem oficial do MySQL executa automaticamente todos os .sql
  # que encontrar em /docker-entrypoint-initdb.d/ na PRIMEIRA vez
  # que o container sobe (quando o volume ainda está vazio).
  # É assim que nossas tabelas são criadas automaticamente!
  # ":ro" significa read-only - o container pode ler mas não escrever
  - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

# Define em qual rede este container vai participar.
# Containers na mesma rede se enxergam pelo nome do serviço.
# "devops-net" é definida na seção "networks:" abaixo.
networks:
  - devops-net

```

```

# Política de reinicialização:
# "unless-stopped" = reinicia automaticamente se cair por erro,
# mas não reinicia se você parar manualmente com "docker compose stop"
restart: unless-stopped

# Healthcheck: o Docker verifica periodicamente se o container
# está realmente funcionando (não apenas "em execução").
# Um container pode estar "rodando" mas com o MySQL ainda inicializando.
# O healthcheck sabe distinguir isso.
healthcheck:
    # O comando executado dentro do container para testar a saúde.
    # mysqladmin ping verifica se o MySQL está aceitando conexões.
    test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1",
           "-u", "devops", "-pdevops"]
    interval: 10s      # Testa a cada 10 segundos
    timeout: 5s        # Se não responder em 5s, conta como falha
    retries: 10        # Precisa de 10 falhas para marcar como "unhealthy"
    start_period: 30s  # Aguarda 30s antes de começar (MySQL demora para iniciar)

```

4.3.2 Serviço 2 — Backend (Node.js)

```

backend:

# "build:" diz ao Docker para CONSTRUIR a imagem a partir de um Dockerfile,
# em vez de baixar uma pronta (diferente do mysql que usa "image:").
build:
    context: ./backend    # Pasta onde está o código e o Dockerfile
    dockerfile: Dockerfile # Nome do arquivo Dockerfile (opcional se for Dockerfile)

# Nome da imagem gerada após o build.
# Na próxima vez que subir, o Docker verifica se já existe essa imagem.
image: devops-site-backend:latest

container_name: devops-site-backend

```

```

# "expose:" torna a porta acessível apenas para outros containers
# na mesma rede Docker - NÃO expõe para o mundo externo (sua máquina).
# Diferença entre expose e ports:
# - ports: "3000:3000" → acessível de fora (sua máquina, internet)
# - expose: "3000" → acessível só internamente (outros containers)
# Usamos expose aqui porque só o NGINX precisa acessar o backend.
expose:
  - "3000"

# Variáveis de ambiente injetadas no container do backend.
# O código Node.js lê essas variáveis com process.env.NOME_DA_VARIAVEL
environment:
  NODE_ENV:      production
  PORT:          "3000"

# DB_HOST aponta para "mysql" - o NOME DO SERVIÇO do container MySQL.
# Dentro de uma rede Docker, o nome do serviço funciona como hostname.
# O Docker tem um DNS interno que resolve "mysql" para o IP do container.
# Você não precisa saber qual é o IP - o Docker cuida disso.
DB_HOST:        mysql
DB_PORT:        "3306"
DB_USER:        devops
DB_PASSWORD:    devops
DB_NAME:        devops_site

# Segredo para assinar os tokens JWT de autenticação.
# Em produção, use um valor longo e aleatório gerado com:
# node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
JWT_SECRET:     troque-este-segreto-em-producao
JWT_EXPIRES_IN: 8h

# Origens permitidas pelo CORS (Cross-Origin Resource Sharing).
# Evita que sites de terceiros façam chamadas para sua API.

```

```

ALLOWED_ORIGINS: "http://localhost,http://localhost:80"

# depends_on define a ORDEM de inicialização dos serviços.
# O backend só vai subir depois que o mysql estiver saudável.
# "condition: service_healthy" usa o resultado do healthcheck -
# não basta o container estar rodando, ele precisa estar com
# o MySQL respondendo de verdade.
depends_on:
  mysql:
    condition: service_healthy

networks:
  - devops-net

restart: unless-stopped

healthcheck:
  test: ["CMD", "wget", "-q0-", "http://localhost:3000/health"]
  interval: 15s
  timeout: 5s
  retries: 5
  start_period: 20s

```

4.3.3 Serviço 3 — Frontend (NGINX)

```

frontend:

# Aqui usamos a imagem oficial do NGINX Alpine diretamente,
# sem Dockerfile próprio. Por quê?
# Porque o frontend é só HTML/CSS/JS estático - não precisa de build.
# Configuramos tudo via volumes (bind mounts).
image: nginx:1.27-alpine

container_name: devops-site-frontend

```

```
# Aqui sim usamos "ports:" porque o NGINX precisa ser acessível
# do seu browser. "80:80" significa:
# porta 80 da sua máquina → porta 80 do container NGINX.
# Quando você abre http://localhost, o browser bate na porta 80
# da sua máquina, que o Docker encaminha para o container NGINX.
ports:
  - "80:80"

# Bind mounts para o NGINX:
volumes:
  # Os arquivos HTML/CSS/JS ficam na pasta ./frontend do projeto.
  # Montamos essa pasta em /usr/share/nginx/html, que é o diretório
  # padrão onde o NGINX busca os arquivos estáticos para servir.
  # ":ro" = read-only (o NGINX só lê, não precisa escrever)
  - ./frontend:/usr/share/nginx/html:ro

  # O arquivo de configuração do NGINX substituindo o padrão.
  # Sem isso, o NGINX usaria a configuração padrão dele,
  # que não sabe fazer proxy para o backend.
  - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro

# depends_on simples (sem condition):
# o frontend sobe depois do backend, mas não espera healthcheck.
# Isso porque o NGINX pode subir mesmo sem o backend estar pronto -
# ele serve os arquivos estáticos de qualquer forma.
# Só as chamadas de API (/api/*) precisam do backend.
depends_on:
  - backend

networks:
  - devops-net

restart: unless-stopped
```

```

# O healthcheck do NGINX usa /nginx-health - um endpoint especial
# configurado no nginx.conf que responde imediatamente sem depender
# do backend. Isso evita um deadlock: o healthcheck do NGINX nunca
# vai falhar por causa do backend estar lento.
healthcheck:
  test: ["CMD", "wget", "-q0-", "http://localhost/nginx-health"]
  interval: 30s
  timeout: 5s
  retries: 3
  start_period: 10s

```

4.3.4 Volumes e Redes

```

# =====
# SEÇÃO: volumes
# Declara os volumes nomeados usados pelos serviços acima.
# Volumes nomeados são gerenciados pelo Docker em:
# /var/lib/docker/volumes/<nome>/
# São mais robustos que bind mounts para dados persistentes.
# =====
volumes:
  mysql_data:      # Persiste os dados do banco MySQL
    driver: local  # Armazena localmente nesta máquina

# =====
# SEÇÃO: networks
# Declara as redes Docker.
# Containers na mesma rede se comunicam pelo nome do serviço.
# Containers em redes diferentes não se enxergam - isolamento!
# =====
networks:
  devops-net:
    driver: bridge  # Rede bridge: o modo padrão e mais comum

```

```
# Cria uma rede virtual privada para os containers
```

5 Redes no Docker — Como os Containers se Comunicam

5.1 O problema sem rede

Sem configuração de rede, cada container é uma ilha. Eles não se enxergam. O backend Node.js não conseguiria chegar no MySQL, e o NGINX não conseguiria encaminhar requisições para o backend.

5.2 A rede bridge

Quando você declara uma rede do tipo **bridge** no Compose e coloca todos os serviços nela, o Docker cria uma rede virtual privada. Dentro dessa rede:

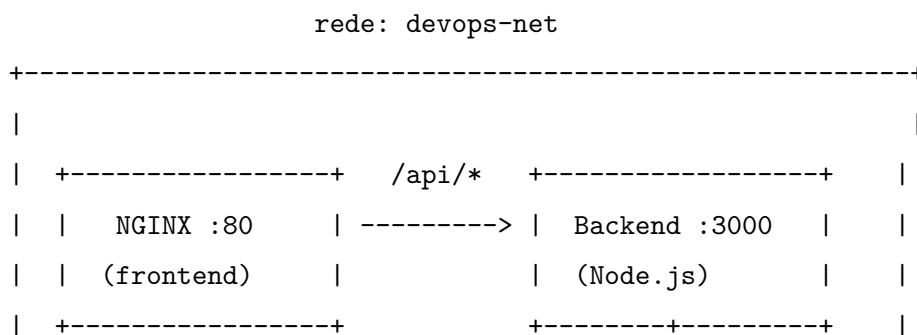
- Cada container tem um IP atribuído automaticamente
- O Docker tem um **servidor DNS interno** que traduz nomes de serviços em IPs
- Containers se comunicam pelo **nome do serviço** — como se fosse um hostname

No projeto, o backend se conecta ao MySQL assim:

```
// backend/lib/db.js
const pool = mysql.createPool({
  host: process.env.DB_HOST, // DB_HOST = "mysql" (nome do serviço)
  port: 3306,
  user: 'devops',
  password: 'devops',
  database: 'devops_site',
});
```

Quando o Node.js tenta resolver `mysql`, o DNS interno do Docker responde com o IP do container MySQL. Simples assim.

5.3 Visualizando a comunicação



```
| | |
| | mysql:3306 |
| | |
| +-----v-----+ |
| | MySQL :3306 | |
| | (banco de dados)| |
| +-----+ |
| |
+-----+
```

Sua máquina (host):

- `http://localhost` → NGINX (porta 80 exposta)
- `localhost:3306` → MySQL (porta 3306 exposta)
- Backend NÃO está acessível do host (só expose, não ports)

6 Volumes — Como os Dados Sobrevivem

6.1 O problema da efemeridade

Containers são efêmeros por design. Quando você para e remove um container, tudo que estava dentro dele some. Para um servidor web isso é ótimo — você quer poder destruir e recriar sem medo. Mas para um banco de dados, isso seria catastrófico.

6.2 Dois tipos de volume no projeto

6.2.1 Volume Nomeado — para dados do banco

```
volumes:  
  - mysql_data:/var/lib/mysql
```

O Docker cria e gerencia uma pasta em `/var/lib/docker/volumes/mysql_data/`. Os dados do MySQL ficam lá, fora do container. Mesmo que você destrua e recrie o container, os dados continuam.

Quando os dados somem? Apenas quando você explicitamente remove o volume:

```
docker compose down -v  # O "-v" remove os volumes também
```

Sem o `-v`, os dados sempre persistem:

```
docker compose down      # Para e remove containers, mas MANTÉM volumes  
docker compose up -d     # Sobe novamente usando os dados existentes
```

6.2.2 Bind Mount — para arquivos do projeto

```
volumes:  
  - ./frontend:/usr/share/nginx/html:ro  
  - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro  
  - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

O bind mount conecta uma pasta/arquivo do seu computador diretamente a um caminho dentro do container. Qualquer mudança no arquivo original aparece instantaneamente dentro do container.

É por isso que, quando você edita um arquivo HTML/CSS/JS no projeto e recarrega o browser, a mudança aparece sem precisar rebuildar a imagem. O NGINX está servindo diretamente da sua pasta local.

7 Ordem de Inicialização e Healthchecks

7.1 O problema do “quem sobe primeiro”

Um dos problemas mais comuns com Docker Compose é a ordem de inicialização. Se o backend tenta se conectar ao MySQL antes dele estar pronto, a conexão falha e o backend cai.

O `depends_on` básico resolve a *ordem* (MySQL inicia antes do backend), mas não resolve o *tempo* (MySQL pode ainda estar inicializando quando o backend tenta conectar).

7.2 A solução: healthcheck + condition

```
backend:
  depends_on:
    mysql:
      condition: service_healthy # Espera o healthcheck passar
```

Com `condition: service_healthy`, o Docker só libera o backend para iniciar quando o healthcheck do MySQL retornar sucesso. O healthcheck do MySQL é:

```
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-u", "devops", "-pdevops"]
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 30s
```

O MySQL demora ~20-30 segundos para inicializar completamente. O `start_period: 30s` faz o Docker esperar esse tempo antes de começar a contar falhas do healthcheck.

7.3 A cadeia de dependências do projeto

```
mysql (healthcheck: mysqladmin ping)
|
| service_healthy
v
backend (healthcheck: wget /health)
|
| depends_on (sem condition)
```

v

```
frontend (healthcheck: wget /nginx-health)
```

O frontend usa `depends_on` simples (sem `condition`) porque o NGINX pode servir os arquivos estáticos mesmo sem o backend. Só as chamadas de API precisam do backend.

8 Comandos Essenciais do Docker Compose

8.1 Subindo e parando

```
# Sobe todos os serviços em background (-d = detached)
docker compose up -d

# Sobe e reconstrói as imagens (necessário após mudar o código do backend)
docker compose up -d --build

# Reconstrói e sobe apenas um serviço específico
docker compose up -d --build backend

# Para todos os containers (mas mantém volumes e redes)
docker compose down

# Para e remove tudo, incluindo volumes (APAGA o banco!)
docker compose down -v
```

8.2 Monitorando

```
# Lista os containers e seus status (Running, healthy, etc.)
docker compose ps

# Acompanha os logs de todos os serviços em tempo real
docker compose logs -f

# Logs de um serviço específico
docker compose logs -f backend
docker compose logs -f mysql

# Ver as últimas 50 linhas de log
docker compose logs --tail=50
```

8.3 Acessando containers

```
# Abre um terminal interativo dentro do container do backend
docker compose exec backend sh

# Executa um comando específico dentro do MySQL
docker compose exec mysql mysql -u devops -pdevops devops_site

# Executa uma query SQL diretamente
docker compose exec mysql mysql -u devops -pdevops devops_site \
    -e "SELECT * FROM users;"
```

8.4 Inspeccionando

```
# Detalhes de um container (IP, volumes, variáveis de ambiente)
docker inspect devops-site-backend

# Uso de recursos em tempo real (CPU, memória, rede)
docker stats

# Listar todos os volumes
docker volume ls

# Listar todas as redes
docker network ls

# Inspeccionar a rede do projeto
docker network inspect devops-site_devops-net
```

8.5 Limpeza

```
# Remove imagens, containers parados e redes não usadas
docker system prune

# Remove TUDO incluindo volumes (CUIDADO: apaga dados!)
```

```
docker system prune -a --volumes
```

```
# Remove apenas imagens não utilizadas
```

```
docker image prune
```

9 Conectando ao Banco de Dados — Como Funciona no Projeto

9.1 A jornada de uma requisição

Quando você faz login no DevOps World, acontece a seguinte sequência:

1. Browser → POST `http://localhost/api/auth/login`
`{"email": "demo@devops.local", "password": "demo1234"}`
2. NGINX recebe a requisição na porta 80
 - + O `nginx.conf` tem uma regra:

```
location /api/ {  
    proxy_pass http://backend; ← encaminha para o container backend  
}
```
3. Backend (Node.js) recebe a requisição na porta 3000
 - + Extrai email e senha do body
 - + Executa uma query SQL no MySQL:

```
SELECT id, email, password, name FROM users WHERE email = ?
```
4. MySQL (container mysql) executa a query
 - + Retorna o usuário encontrado
5. Backend verifica a senha, gera o JWT
 - + Responde ao NGINX com `{token, user}`
6. NGINX devolve a resposta ao Browser

9.2 Como o backend se conecta ao MySQL

No arquivo `backend/lib/db.js`:

```
const mysql = require('mysql2/promise');  
  
// Cria um "pool" de conexões  
// Pool = um conjunto de conexões abertas e reutilizáveis  
// Em vez de abrir e fechar uma conexão a cada query (lento),
```

```
// o pool mantém conexões abertas e as reutiliza (rápido)
const pool = mysql.createPool({
  host:      process.env.DB_HOST,      // "mysql" (nome do serviço Docker)
  port:      process.env.DB_PORT,      // 3306
  user:      process.env.DB_USER,      // "devops"
  password:  process.env.DB_PASSWORD,  // "devops"
  database:  process.env.DB_NAME,      // "devops_site"

  connectionLimit: 10, // Máximo de 10 conexões simultâneas
  timezone: 'Z',       // Armazena datas em UTC (padrão universal)
});
```

As variáveis `process.env.*` vêm exatamente das variáveis definidas no `docker-compose.yml` na seção `environment:` do serviço `backend`.

9.3 Como o banco é inicializado automaticamente

A imagem oficial do MySQL tem um comportamento especial: na primeira vez que ela sobe com um volume vazio, ela executa automaticamente todos os arquivos `.sql` que encontrar em `/docker-entrypoint-initdb.d/`.

No compose, fizemos:

```
volumes:
  - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

Isso faz o MySQL executar nosso `init.sql` automaticamente, criando as tabelas e inserindo os dados demo. Você não precisa fazer nada manualmente.

Atenção: isso só acontece na **primeira vez** (quando o volume está vazio). Se você quiser que o `init.sql` rode novamente, precisa destruir o volume:

```
docker compose down -v
docker compose up -d
```

10 Variáveis de Ambiente — A Forma Segura de Configurar

10.1 Por que não colocar credenciais no código

Imagine que você coloca a senha do banco diretamente no código:

```
// NUNCA faça isso!
const pool = mysql.createPool({
  password: 'minhasenha123',
});
```

Se você commitar esse código no GitHub, a senha fica exposta para o mundo inteiro. E mesmo que o repositório seja privado, qualquer pessoa com acesso ao repositório vê a senha.

10.2 A solução: variáveis de ambiente

Variáveis de ambiente são valores configurados **fora do código**, no ambiente de execução. O código apenas lê esses valores:

```
// Correto: lê do ambiente de execução
const pool = mysql.createPool({
  password: process.env.DB_PASSWORD,
});
```

No Docker Compose, você injeta essas variáveis na seção `environment::`

```
environment:
  DB_PASSWORD: devops
```

10.3 O arquivo `.env.example`

O projeto tem um arquivo `.env.example` que documenta todas as variáveis necessárias sem revelar os valores reais:

```
# Copie para .env e preencha com seus valores
DB_HOST=mysql
DB_PORT=3306
DB_USER=devops
DB_PASSWORD=COLOQUE_SUA_SENHA_AQUI
DB_NAME=devops_site
```

```
JWT_SECRET=GERE_UM_SEGREDO_FORTE_AQUI
```

O arquivo `.env` real está no `.gitignore` — nunca vai para o repositório.

11 O nginx.conf — Como o NGINX Integra Tudo

O NGINX tem um papel central: ele é o único serviço acessível do mundo externo (porta 80). Tudo passa por ele.

```
# Define o usuário com o qual o NGINX roda dentro do container
user  nginx;

# "auto" = usa todos os núcleos de CPU disponíveis
worker_processes  auto;

events {
    worker_connections  1024;  # Conexões simultâneas por worker
}

http {
    # Inclui os tipos MIME padrão (text/html, image/png, etc.)
    include /etc/nginx/mime.types;

    # Ativa compressão gzip - reduz o tamanho dos arquivos transferidos
    # Um arquivo JS de 100KB pode virar 30KB comprimido
    gzip on;
    gzip_types text/css application/javascript application/json;

    # Define o container backend como destino do proxy
    # "backend" é o nome do serviço no Docker Compose
    # O DNS interno do Docker resolve "backend" para o IP do container
    upstream backend {
        server backend:3000;
        keepalive 32;  # Mantém 32 conexões abertas reutilizáveis
    }

    server {
        listen 80;          # Escuta na porta 80 (HTTP)
        server_name _;      # Aceita qualquer hostname
    }
}
```

```

# Diretório dos arquivos estáticos (montado via volume)
root /usr/share/nginx/html;
index index.html;

# Endpoint de healthcheck PRÓPRIO do NGINX
# Não depende do backend - responde imediatamente
# Usado pelo healthcheck do Docker Compose
location = /nginx-health {
    return 200 "ok\n";
}

# Regra de proxy: tudo que começa com /api/ vai para o backend
# proxy_pass encaminha a requisição para o container Node.js
location /api/ {
    proxy_pass http://backend;

    # Headers que informam ao backend o IP real do cliente
    proxy_set_header X-Real-IP      $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header Host           $host;
}

# SPA Fallback: qualquer rota desconhecida devolve o index.html
# Isso permite que o JavaScript do frontend gerencie as rotas
# Sem isso, acessar /projetos diretamente daria 404
location / {
    try_files $uri $uri/ /index.html;
}
}
}

```

12 Fluxo Completo — Do docker compose up ao Site Funcionando

Vamos ver exatamente o que acontece quando você executa `docker compose up -d --build`:

1. **Docker lê o `docker-compose.yml`** Identifica 3 serviços, 1 volume nomeado e 1 rede.
2. **Docker cria a rede `devops-net`** Uma rede bridge virtual é criada. Os containers vão se comunicar por ela.
3. **Docker faz o build do backend** Lê o `backend/Dockerfile`, executa os estágios e gera a imagem `devops-site-backend:latest`.
4. **Docker inicia o container MySQL** - Cria o volume `mysql_data` se não existir - Monta o `init.sql` em `/docker-entrypoint-initdb.d/` - Na primeira vez: executa o SQL (cria tabelas, insere dados demo) - Começa o healthcheck a cada 10 segundos - Aguarda o MySQL ficar pronto (~20-30s)
5. **MySQL healthcheck passa → Docker inicia o Backend** - Container `devops-site-backend` sobe - Node.js lê as variáveis de ambiente do Compose - `lib/db.js` cria o pool de conexões apontando para `mysql:3306` - Healthcheck verifica `GET /health` a cada 15s
6. **Backend está de pé → Docker inicia o Frontend** - Container `devops-site-frontend` (NGINX) sobe - Monta `./frontend` em `/usr/share/nginx/html` - Monta `./nginx/nginx.conf` na config do NGINX - NGINX começa a servir o site na porta 80
7. **Você abre `http://localhost`** - Browser → NGINX → serve `index.html` - JavaScript carrega, faz `GET /api/projects` - NGINX encaminha para `backend:3000` - Backend faz query no MySQL - Resposta volta pelo mesmo caminho

Tudo isso acontece automaticamente. Um único comando.

13 Resumo dos Conceitos

Conceito	O que é	Exemplo no projeto
Imagem	Molde somente leitura	<code>mysql:8.4</code> , <code>nginx:1.27-alpine</code> , <code>devops-site-backend:latest</code>
Container	Imagem em execução	<code>devops-site-mysql</code> , <code>devops-site-backend</code> , <code>devops-site-frontend</code>
Dockerfile	Receita para criar imagem	<code>backend/Dockerfile</code>
docker- compose.yml	Orquestração de múltiplos containers	<code>docker-compose.yml</code> na raiz
Volume nomeado	Dados persistentes gerenciados pelo Docker	<code>mysql_data</code>
Bind mount	Pasta/arquivo do host mapeado no container	<code>./frontend:/usr/share/nginx/html</code>
Rede bridge	Rede virtual privada entre containers	<code>devops-net</code>
Healthcheck	Verificação periódica de saúde	<code>mysqladmin ping</code> , <code>wget /health</code>
depends_on	Ordem de inicialização	<code>backend</code> depende do <code>mysql</code>
ports	Expõe porta para o mundo externo	<code>80:80</code> (NGINX), <code>3306:3306</code> (MySQL)
expose	Expõe porta apenas internamente	<code>3000</code> (backend)
environment	Variáveis de ambiente injetadas	<code>DB_HOST</code> , <code>JWT_SECRET</code>

14 Referência Rápida de Comandos

```
# -- Subir -----
docker compose up -d                # Sobe em background
docker compose up -d --build        # Sobe e reconstrói imagens
docker compose up -d --build backend # Reconstrói só o backend

# -- Parar -----
docker compose down                # Para e remove containers
docker compose down -v            # Para + remove volumes (apaga banco)
docker compose stop                # Apenas para (não remove)
docker compose start               # Inicia containers parados

# -- Monitorar -----
docker compose ps                  # Status dos containers
docker compose logs -f             # Logs em tempo real
docker compose logs -f backend     # Logs de um serviço
docker compose logs --tail=50      # Últimas 50 linhas

# -- Acessar -----
docker compose exec backend sh      # Terminal no backend
docker compose exec mysql mysql \
  -u devops -pdevops devops_site    # MySQL interativo

# -- Inspeccionar -----
docker compose ps                  # Status
docker stats                       # CPU e memória em tempo real
docker volume ls                   # Lista volumes
docker network ls                   # Lista redes

# -- Limpar -----
docker system prune                 # Remove recursos não usados
docker image prune                  # Remove imagens não usadas
```

15 Próximos Tópicos para Estudar

Agora que você entende o Docker e o Compose, os próximos passos naturais são:

Docker avançado - Multi-stage builds mais complexos - Docker secrets (para senhas em produção)
- Registry privado (ECR da AWS)

Terraform O próximo tema do DevOps World. Terraform usa arquivos `.tf` para descrever infraestrutura na AWS (EC2, RDS, VPC) da mesma forma que o Docker Compose descreve containers.

AWS Onde esse projeto vai ao ar de verdade: - EC2 para hospedar os containers - RDS para o MySQL gerenciado - ECS/EKS para orquestrar containers em produção - ECR para armazenar as imagens Docker

CI/CD com GitHub Actions Automatizar o processo de build, test e deploy usando workflows `.yaml` — que têm uma sintaxe muito parecida com o Docker Compose.

Este documento foi criado especificamente para o seu estudo, usando o projeto DevOps World como base real. Todos os exemplos de código são do projeto em produção.